INSTRUCTIONS

Please read the instructions carefully before doing the questions.

- You can use materials in your computer, notebook and text book.

- You are **NOT allowed** to use any device to share data with others.

Beside the above conditions, students must follow the following requirements:

1. The work must complete by using Visual Studio 2022++

2. The Framework must be .NET 8.0

3. THIS PART IS VERY IMPORTANT, PLEASE READ IT CAREFULLY AND FOLLOW THE INSTRUCTIONS.

- You are given a database script (.sql file) in Zip file. Execute the script before doing questions.
- **You must use the given solution.**
- **You are not allowed to add any more libraries via NuGet Package Manager into given solution.**
- **Submission Guideline:**

  Submit your work for each question separately. For each question, please:

  - Publish your project using the command:

    ```
    dotnet publish -c Release -o ./[QuestionNumber_StudentAccount]
    ```

    Example:

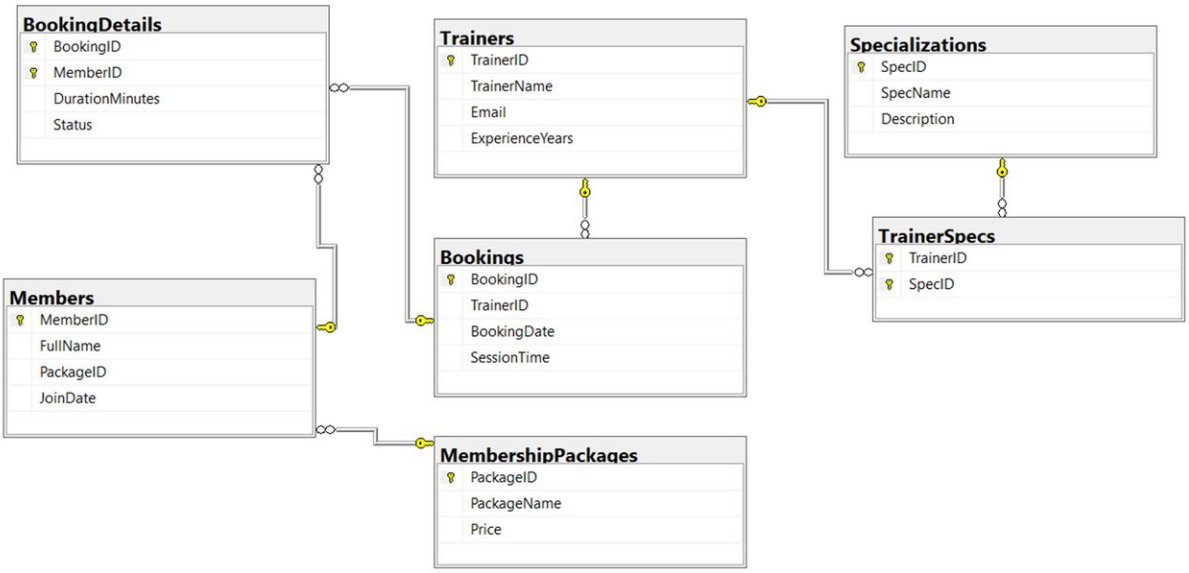
    ```
    dotnet publish -c Release -o ./Q1_trungnthe123432
    ```

  - Submit the root folder of the project into the PEA_Client application.
  - If the root folder of the project is too large, you may delete the following subfolders to reduce its size before submitting: /bin, /obj

Just one of above requirements is violated, your work will be considered as invalid.

Question 1:

You are provided with a database as following diagram



Ensure that the database connection string is adjustable through **appsettings.json** (otherwise your post will get 0 point) as pattern:

```
{
    "ConnectionStrings":
    {
        "MyCnn":""
    }
}
```

Your task is to implement the following APIs using **.NET 8** and **Entity Framework Core**:

Method	URL	Description
GET	`/api/MembershipPackages`	Retrieve all membership packages with registered member count statistics. Supports JSON.
GET	`/api/filter-members`	Advanced search for members with training duration stats and recent coach assignment logs.
POST	`/api/MembershipPackages`	Add a new membership package to the gym catalog.
DELETE	`/api/MembershipPackages/{packageId}`	Remove a membership package from the system.

1. GET /api/MembershipPackages

- Retrieve a list of all membership packages from the database.

- Response Fields:

`PackageId` (int)

`PackageName` (string)

`Price` (decimal)

`MemberCount` (int): Total number of gym members currently registered under this membership package.

Request URL

```
http://localhost:5000/api/MembershipPackages
```

Server response

Code	Details
200	Response body


```
[
  {
    "packageId": 1,
    "packageName": "Goi Thang Co Ban",
    "price": 300000,
    "memberCount": 1
  },
  {
    "packageId": 2,
    "packageName": "Goi Binh Minh",
    "price": 450000,
    "memberCount": 1
  },
  {
    "packageId": 3,
    "packageName": "Goi Chuyen Nghiep",
    "price": 800000,
    "memberCount": 1
  },
  {
    "packageId": 4,
    "packageName": "Goi Gia Dinh",
    "price": 1500000,
    "memberCount": 1
  },
```


2. GET /api/filter-members

Retrieve a list of gym members with advanced booking performance statistics.

- **Filtering Logic**:

  `packageId` (int, optional): If specified, return only members registered with this Package ID.

  `minDuration` (int, optional): If specified, return only members who have booked sessions with a duration greater than or equal to this value.

- **Response Fields**:

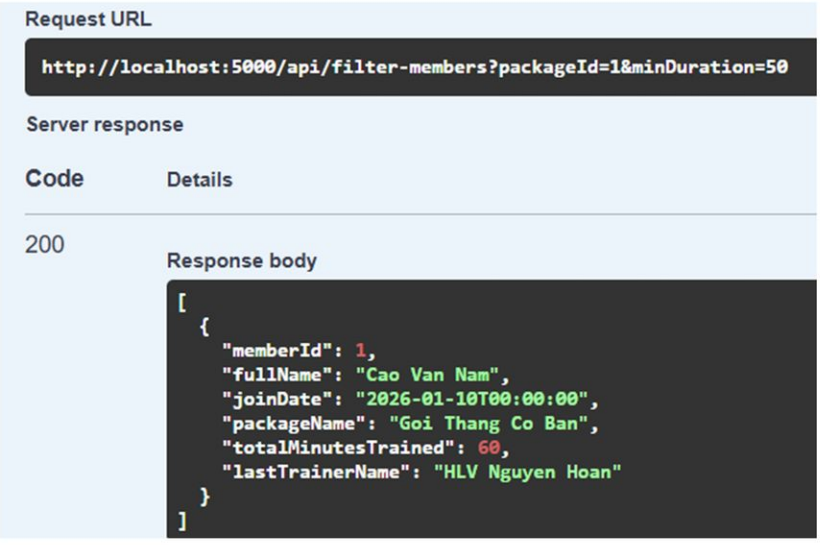
  `MemberId`, `FullName`, `JoinDate`, `PackageName`.

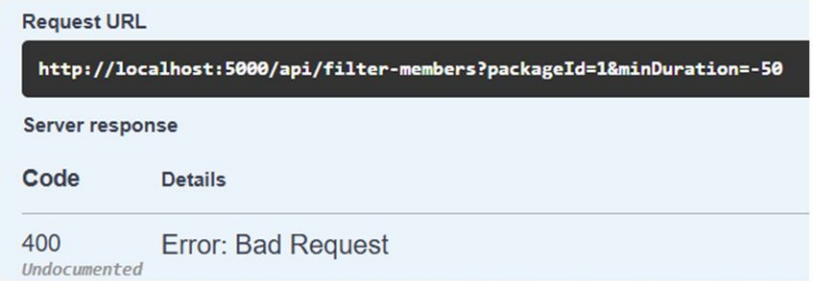
  `TotalMinutesTrained` (int): The total sum of `DurationMinutes` across all booking sessions associated with this member in the `BookingDetails` table. Return **0** if there are no sessions.

  `LastTrainerName` (string): The `TrainerName` of the personal trainer who coached this member most recently (based on `Bookings.SessionTime` descending). If the member has never booked any sessions, return "**No Trainers**".

- **Error handling**: Return **400 Bad Request** if `minDuration` is negative (if provided).

- **Sorting**: Sort results by `JoinDate` in **descending** order (newest members first).



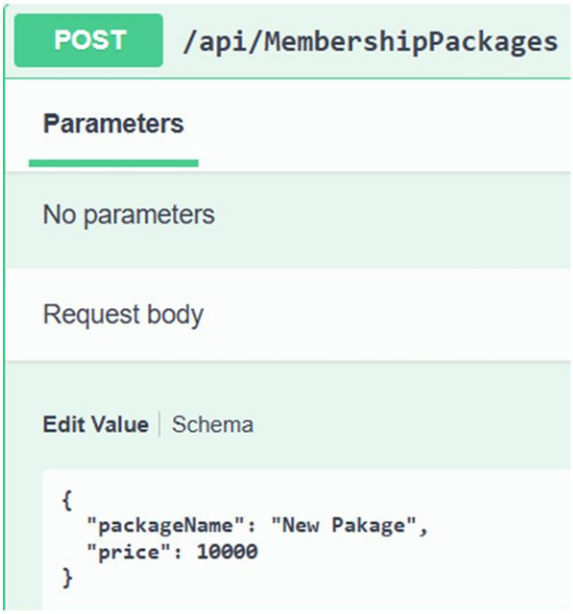


3. POST /api/MembershipPackages

Accept `PackageName` and `Price` inside the request body.

- **Return**: **201 Created** along with the complete newly created `MembershipPackage` entity (ensure navigation properties like `Members` are returned as empty lists `[]` instead of `null` to avoid client integration crashes).

- **Error handling**: If the package `PackageName` already exists (case-insensitive check), return **400 Bad Request** with message `"Membership package name already exists"`.



Request URL

```
http://localhost:5000/api/MembershipPackages
```

Server response

Code	Details
201 *Undocumented*	Response body

```
{
  "packageId": 7,
  "packageName": "New Pakage",
  "price": 10000,
  "members": []
}
```

POST `/api/MembershipPackages`

Parameters

No parameters

Request body

Edit Value | Schema

```
{
  "packageName": "Goi Gia Dinh",
  "price": 10000
}
```

Request URL

```
http://localhost:5000/api/MembershipPackages
```

Server response

Code	Details
400 *Undocumented*	Error: Bad Request Response body

```
Membership package name already exists
```

4. DELETE /api/MembershipPackages/{packageId}

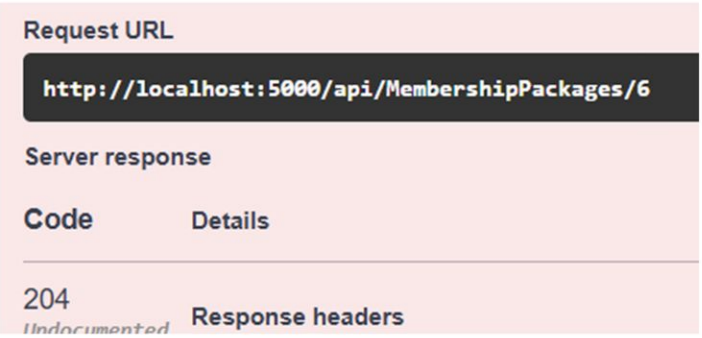
Delete the package with the specified `packageId` from the database.

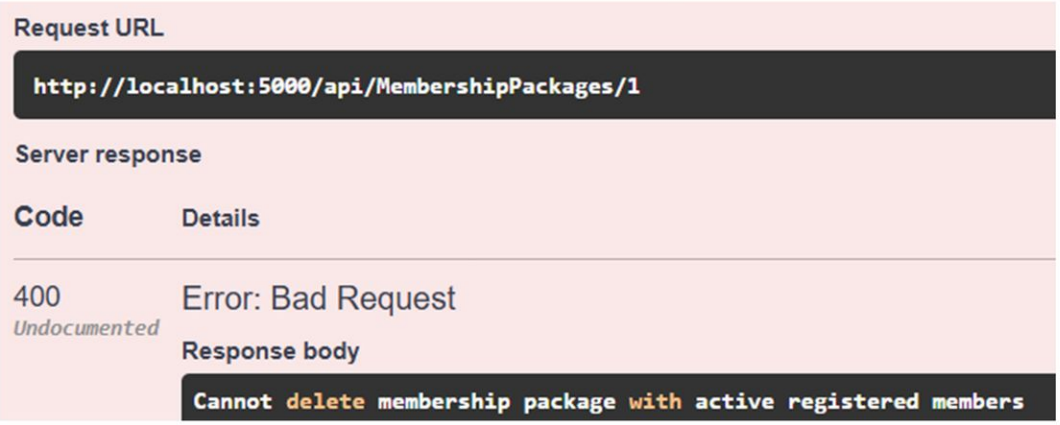
- Return: **204 No Content** if the operation is successful.

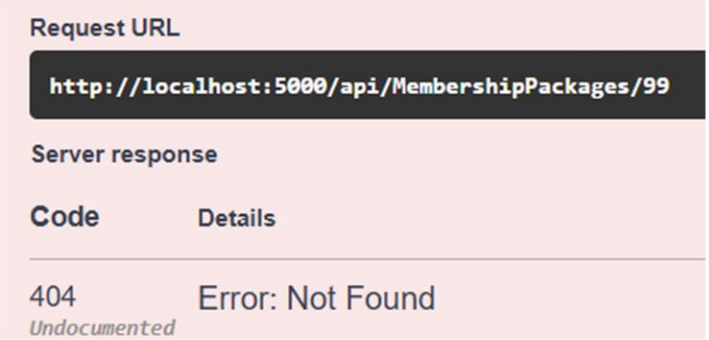
- Error handling:

If there are still gym members registered under this package (i.e. records exist in the `Members` table referencing this `PackageID`), return **400 Bad Request** with message `"Cannot delete membership package with active registered members"`.

If the `packageId` does not exist in the database, return a **404 Not Found** error.







Question 2:

In this question, you are asked to write an MVC/Razor Pages application. The application fetch data by calling pre-existing RESTful API hosted at GivenAPIBaseUrl. The API are provided in a separate project named GivenAPIs, which students must run locally to start the API server

1. Important Notes

- Students **MUST** use HttpClient to call the API.

- The GivenAPIBaseUrl must be written in appsettings.json

```
{
    "GivenAPIBaseUrl": "http://localhost:5100"
}
```

- Students must use **explicit string concatenation** for API endpoints (e.g., baseUrl + "/endpoint").

- All input and output elements in the HTML source **must** have an **'id'** attribute to ensure accessibility and traceability (Students can refer to the html files in given materials, which provides code snippets for the assignment).

2. Provided APIs

GET /api/specializations : Returns a list of all available fitness specializations (SpecID, SpecName).

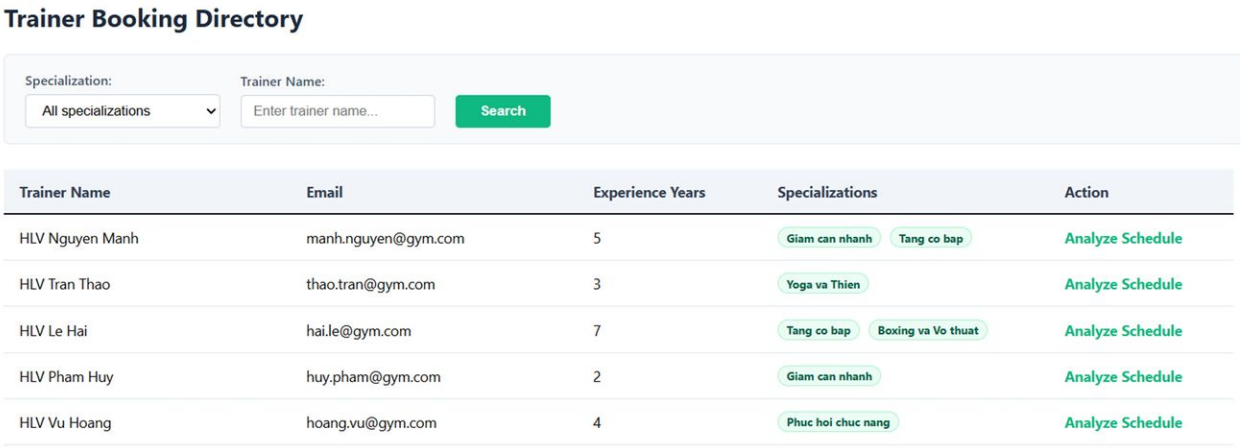
GET /api/trainers/search?specId={id}&trainerName={name} : Returns a list of trainers filtered by a specific specialization (exact match) and trainer name (partial match).

GET /api/trainers/{trainerId} : Returns detailed information of a trainer, their specializations, and their scheduled member booking training sessions.

3. Requirements

3.1. Trainer Booking Directory Page

- URL : /Trainer



- Search Area :

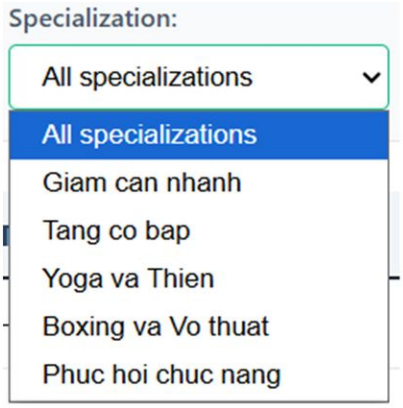
A **Dropdown (Select)** for **Specializations** (id: **sl_specializations**).

The first option **"All specializations"** is set by **default**, must have id **op_0**, using the <option> tag.

Each specific specialization option must have id **op_{specId}**.

An **Input (Text)** for **Trainer Name** (id: **ip_trainerName**) to search by full name or partial name.

A **Search** button (id: **bt_search**) to filter the table below.



- **Data Table** : Display matching trainers with columns:

  Trainer Name : Display name with id **td_name_{TrainerID}**.

  Email : Display email with id **td_email_{TrainerID}**.

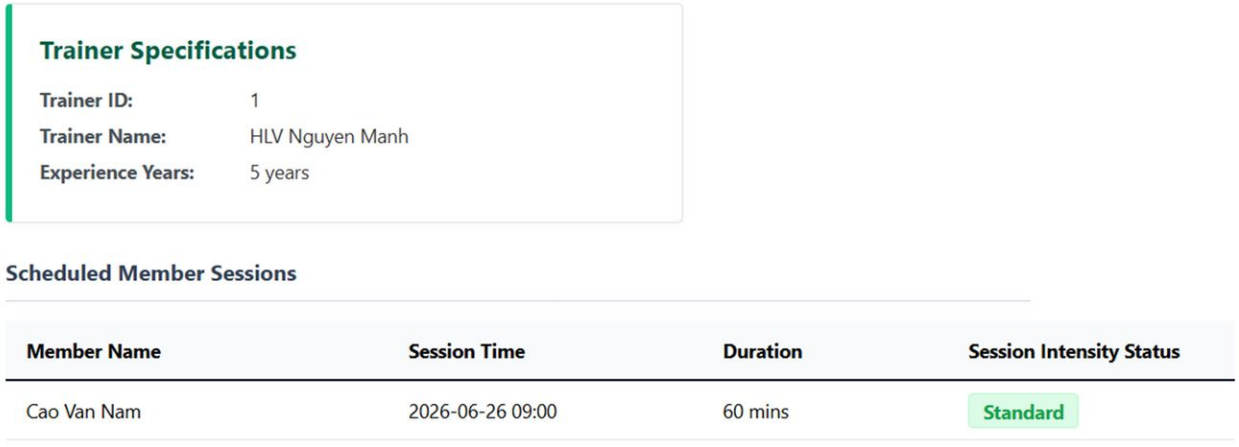
  Experience Years : Display years with id **td_exp_{TrainerID}**.

  Specializations : Each specialization associated with the trainer displayed as a with id **span_{SpecName}_{SpecID}**.

  Action : A link **"Analyze Schedule"** (id: **a_{TrainerID}**) redirecting to /Trainer/Analyze/{TrainerID}.

3.2. Trainer Schedule Analysis Page

- **URL** : /Trainer/Analyze/{TrainerID}



- **Display basic information** :

Trainer ID : Display trainer ID inside a with id **span_{TrainerID}**.

Trainer Name : Display full name inside a with id **span_trainerName**.

Experience Years : Display experience years inside a with id **span_experienceYears**.

- Display Scheduled Members Sessions : A table showing member sessions booked with this trainer, including:

Member Name : Display member's full name with id **td_member_{MemberID}**.

Session Time : Display session date-time with id **td_time_{BookingID}** (format: yyyy-MM-dd HH:mm).

Duration : Display session duration in minutes with id **td_duration_{BookingID}_{MemberID}** (e.g., 60 mins).

Session Intensity Status :

Calculate: Intensity = DurationMinutes.

If DurationMinutes > 60: Display **"High Intensity"** (Color: Red, Font-weight: bold).

Otherwise: Display **"Standard"** (Color: Green).

The cell must have id **td_intensity_{BookingID}_{MemberID}**.

4. Summary of HTML Elements ID (with Examples)

Page	Element	Tag	Id	Example
/Trainer	Dropdown Spec	<select>	sl_specializations	<select id="sl_specializations">...</select>
	Spec Option	<option>	op_{SpecId}	<option id="op_2">Yoga va Thien</option>
	Input Trainer Name	<input>	ip_trainerName	<input id="ip_trainerName" type="text" />

	Button Search	<button>	bt_search	<button id="bt_search">Search</button>
	Cell: Trainer Name	<td>	td_name_{TrainerID}	<td id="td_name_1">HLV Nguyen Manh</td>
	Each Spec tag		span_{SpecName}_{SpecID}	Giam can nhanh
	Analyze Link	<a>	a_{TrainerID}	Analyze Schedule
/Trainer/Analyze/{TrainerID}	Field TrainerId		span_{TrainerID}	1
	Cell: Member Name	<td>	td_member_{MemberID}	<td id="td_member_3">Cao Van Nam</td>
	Cell: Intensity Status	<td>	td_intensity_{BookingID}_{MemberID}	<td id="td_intensity_5_3">High Intensity</td>

5. Summary of Required URLs

Function	**URL**
List trainers, filter by specialization and partial name input	/Trainer
View detailed trainer info and member session schedule	/Trainer/Analyze/{TrainerID}